Kubernetes + Helm + Dorarion

> 베타 DB 안내 (2026-10-10): 현재 core는 두 개의 통합 마이그레이션으로 설치합니다. 이전 베타 DB는 새 빈 DB를 쓰거나 기존 데이터를 삭제하고 명시적으로 재설치해야 합니다. SQLx 이력만 삭제하지 마세요. 현재 소스로 새 이미지를 빌드하세요. 최신 정책은 README를 확인하세요. 이전에 다운로드한 PDF에는 과거 마이그레이션 절차가 남아 있을 수 있습니다.

루트 경로의 전용 HTTPS 호스트 이름을 사용하세요. 하위 경로 설치는 지원하지 않습니다. 일회용 비암호화 PostgreSQL 예제에만 아래 Helm 인자로 비암호화 DB 연결을 명시적으로 허용합니다. 운영 외부 DB에는 sslmode=verify-full과 승인된 CA가 필요합니다.

이 가이드는 현재 소스 체크아웃과 일치합니다. 설치 전에 일치하는 core 이미지를 빌드하고 검증하세요. 이전 게시된 차트와 이미지에는 이 수정이 없습니다. PostgreSQL 16 이상이 필요합니다.

간단한 클러스터 설치 가이드

> **목표:** 현재 체크아웃의 Helm 차트로 Dorarion을 설치하고, 외부 PostgreSQL 데이터베이스에 연결하며, 필요한 Kubernetes Secret과 첫 번째 관리자 계정을 만든 다음 애플리케이션을 확인합니다.

- Kubernetes 클러스터에 접근하도록 설정된 터미널에서 모든 명령을 실행합니다.
- 업데이트된 `dorarion-helm` 체크아웃의 루트에서 실행하세요.
- Dorarion에는 Kubernetes 1.28 이상과 Helm 3이 필요합니다.
- 이 차트는 PostgreSQL을 설치하지 않습니다. 먼저 데이터베이스와 백업을 준비하세요.
- PDF 버전도 제공됩니다.

1. Kubernetes 및 Helm 확인

무언가를 생성하기 전에 `kubectl`이 올바른 클러스터를 가리키는지 확인합니다.

```
kubectl config current-context
kubectl cluster-info
kubectl version
helm version
```

컨텍스트가 다른 클러스터를 가리키면 계속 진행하지 마세요. Kubernetes 서버 버전은 최소 1.28이어야 합니다.

2. 로컬 차트와 일치하는 이미지 확인

현재 체크아웃을 확인하고 일치하는 core 소스에서 빌드한 검증된 이미지를 선택합니다.

```
helm show chart .

read -r -p "Verified matching image repository: " DORARION_IMAGE_REPOSITORY
read -r -p "Verified image digest (sha256:...): " DORARION_IMAGE_DIGEST
```

차트 메타데이터는 다음과 같습니다.

- 차트: `dorarion`
- 차트 버전: `0.0.3`
- 애플리케이션 버전: `0.0.3`
- 애플리케이션 이미지: 검증된 저장소와 변경 불가능한 digest.

3. PostgreSQL 준비

빈 PostgreSQL 데이터베이스와 Dorarion 전용 사용자를 만듭니다. 이 사용자는 접속할 수 있어야 하며 테이블, 인덱스, 제약 조건을 생성하고 변경하고 Dorarion 마이그레이션을 실행할 권한이 있어야 합니다.

데이터베이스 제공자가 요구하는 TLS 옵션을 연결 URL에 적용하세요. 일반적인 URL 형식은 다음과 같습니다.

```
postgres://USER:PASSWORD@HOST:5432/DB_NAME?sslmode=verify-full
```

Dorarion을 업그레이드하기 전에 항상 데이터베이스를 백업하세요. 별도의 환경에서 복원을 시험해야 합니다. Helm 차트는 PostgreSQL을 운영하거나 백업하지 않습니다.

선택 사항: 간단한 PostgreSQL Pod 실행

PostgreSQL이 아직 없다면 이 저장소의 단일 노드 PostgreSQL 예제를 사용할 수 있습니다. 이 예제는 10 Gi 영구 볼륨 클레임, PostgreSQL Deployment 하나, 내부 ClusterIP Service를 생성합니다.

> **이 예제는 학습 및 소규모 테스트 설치용입니다.** 고가용성이 아니며, TLS와 백업을 구성하지 않습니다. 프로덕션에서는 관리형 데이터베이스나 지속적으로 유지보수되는 PostgreSQL Operator를 사용하세요.

네임스페이스와 임의의 PostgreSQL 비밀번호를 생성합니다.

```
kubectl create namespace dorarion --dry-run=client -o yaml | kubectl apply -f -

POSTGRES_PASSWORD="$(openssl rand -hex 32)"

kubectl -n dorarion create secret generic dorarion-postgresql \
  --from-literal=POSTGRES_DB=dorarion \
  --from-literal=POSTGRES_USER=dorarion \
  --from-literal=POSTGRES_PASSWORD="$POSTGRES_PASSWORD"
```

체크아웃의 PostgreSQL 예제를 배포하고 준비될 때까지 기다립니다.

```
kubectl apply -f examples/postgresql.yaml

kubectl -n dorarion rollout status deployment/dorarion-postgresql \
  --timeout=5m
kubectl -n dorarion get pod,service,persistentvolumeclaim \
  -l app.kubernetes.io/name=dorarion-postgresql
```

PostgreSQL을 만든 뒤 새 터미널을 열었다면, 화면에 표시하지 않고 비밀번호를 셸 변수로 다시 가져옵니다.

```
POSTGRES_PASSWORD="$(kubectl -n dorarion get secret dorarion-postgresql \
  -o jsonpath='{.data.POSTGRES_PASSWORD}' | base64 --decode)"
```

이 예제를 사용한다면 다음 절에서 이용할 데이터베이스 URL을 설정합니다.

```
DATABASE_URL="postgres://dorarion:${POSTGRES_PASSWORD}@dorarion-
postgresql.dorarion.svc.cluster.local:5432/dorarion"
DORARION_DEMO_HELM_ARGS=(--set extraEnv[0].name=DORARION_ALLOW_INSECURE_DATABASE --set-
string extraEnv[0].value=true)
```

4. 네임스페이스 및 Secret 생성

데이터베이스 마이그레이션이 pre-install hook으로 실행되므로 `helm install` 전에 Secret이 있어야 합니다.

네임스페이스를 생성합니다.

```
kubectl create namespace dorarion --dry-run=client -o yaml | kubectl apply -f -
```

애플리케이션 시크릿을 생성하고 Kubernetes Secret을 만듭니다. PostgreSQL 예제를 사용했다면 `DATABASE_URL`이 이미 설정되어 있습니다. 외부 데이터베이스를 사용한다면 다음 값을 실제 연결 정보로 바꿔 먼저 실행하세요.

```
DATABASE_URL='postgres://USER:PASSWORD@HOST:5432/DB_NAME?sslmode=verify-full'
DORARION_DEMO_HELM_ARGS=()
```

이제 애플리케이션 Secret을 생성합니다.

```
: "${DATABASE_URL:?Set DATABASE_URL before continuing}"
JWT_SECRET="$(openssl rand -hex 48)"
SECRET_ENCRYPTION_KEY="$(openssl rand -base64 32)"
DORARION_BOOTSTRAP_PASSWORD="$(openssl rand -hex 24)"

kubectl -n dorarion create secret generic dorarion-secrets \
  --from-literal=DATABASE_URL="$DATABASE_URL" \
  --from-literal=JWT_SECRET="$JWT_SECRET" \
  --from-literal=SECRET_ENCRYPTION_KEY="$SECRET_ENCRYPTION_KEY" \
  --from-literal=DORARION_BOOTSTRAP_PASSWORD="$DORARION_BOOTSTRAP_PASSWORD"
```

```
unset DATABASE_URL POSTGRES_PASSWORD JWT_SECRET SECRET_ENCRYPTION_KEY
DORARION_BOOTSTRAP_PASSWORD
```

> **이 값을 안전하게 보호하세요.** Git에 커밋하거나 Helm values에 넣거나 채팅, 스크린샷, 티켓에 게시하지 마세요. `SECRET_ENCRYPTION_KEY`는 비밀번호 관리자나 시크릿 관리자에 별도로 백업하세요. 이 키를 잃으면 Dorarion에 직접 저장한 연동 자격 증명을 복구할 수 없습니다.

값을 출력하지 않고 Secret의 키 이름만 확인합니다.

```
kubectl -n dorarion describe secret dorarion-secrets
```

`DATABASE_URL`, `JWT_SECRET`, `SECRET_ENCRYPTION_KEY`, `DORARION_BOOTSTRAP_PASSWORD`가 있어야 합니다.

5. Dorarion 설치 및 첫 번째 관리자 생성

사용자와 CI Runner가 실제로 접근할 HTTPS 주소를 선택합니다. 첫 설치에서만 관리자 bootstrap을 활성화하여 Dorarion을 설치합니다.

```
helm upgrade --install dorarion \
  . \
  "${DORARION_DEMO_HELM_ARGS[@]}" \
  --namespace dorarion \
  --set-string image.repository="$DORARION_IMAGE_REPOSITORY" \
  --set-string image.digest="$DORARION_IMAGE_DIGEST" \
  --set publicUrl=https://dorarion.example.com \
  --set bootstrapAdmin.enabled=true \
  --set bootstrapAdmin.username=admin \
  --set bootstrapAdmin.email=admin@example.com \
  --wait \
  --timeout 15m
```

사용자, GitLab Runner, 보고서나 webhook을 Dorarion으로 보내는 시스템에서 접근할 수 있는 주소를 사용하세요. 프로덕션 설치에서는 HTTPS를 사용해야 합니다.

설치는 다음 순서로 진행됩니다.

1. 차단 방식의 데이터베이스 migration Job 실행
2. Dorarion Deployment와 Service 생성
3. 애플리케이션이 준비될 때까지 대기
4. 첫 번째 관리자 bootstrap Job 실행

6. 관리자 비밀번호 저장

bootstrap은 미리 만든 Secret의 DORARION_BOOTSTRAP_PASSWORD(16자 이상)를 읽으며 비밀번호를 출력하지 않습니다. 셸 변수를 지우기 전에 비밀번호 관리자에 저장하세요. 로그인 시 비밀번호 관리자 또는 Secret에서 가져

옵니다. 확인 후 완료된 Job을 삭제할 수 있습니다. 기존 설치에서 bootstrap을 다시 활성화하지 마세요.

```
kubectl -n dorarion delete job dorarion-bootstrap-admin
```

7. 설치 확인

Helm 릴리스와 Kubernetes 워크로드를 확인합니다.

```
helm status dorarion --namespace dorarion
kubectl -n dorarion get deployment,pods,service,jobs
kubectl -n dorarion rollout status deployment/dorarion
```

마이그레이션이 실패하면 남아 있는 Job과 로그를 확인합니다.

```
kubectl -n dorarion get jobs
kubectl -n dorarion logs job/dorarion-migrate
```

가장 흔한 원인은 잘못된 `DATABASE_URL`, 차단된 PostgreSQL 네트워크 접근, 누락된 Secret 키, 부족한 데이터베이스 권한입니다.

8. 로컬에서 Dorarion 열기

Ingress 없이 빠르게 확인하려면 ClusterIP Service를 포트 포워딩합니다.

```
kubectl -n dorarion port-forward service/dorarion 3000:80
```

이 터미널을 열어 둔 채 다음 주소로 이동합니다.

http://127.0.0.1:3000/login

`admin` 또는 `admin@example.com`과 6단계에서 저장한 비밀번호로 로그인합니다. 다른 터미널에서 readiness를 확인할 수 있습니다.

```
curl --fail http://127.0.0.1:3000/health/ready
```

9. Ingress 구성

Ingress Controller와 DNS가 미리 준비되어 있어야 합니다. 시크릿이 없는 로컬 values 파일 `values.local.yaml`을 만듭니다.

```
publicUrl: https://dorarion.example.com

ingress:
```

```
  enabled: true
  className: nginx
  hosts:
    - host: dorarion.example.com
      paths:
        - path: /
          pathType: Prefix
  tls:
    - secretName: dorarion-tls
      hosts:
        - dorarion.example.com
```

관리자 bootstrap을 활성화하지 않고 적용합니다.

```
helm upgrade dorarion \
  . \
  "${DORARION_DEMO_HELM_ARGS[@]}" \
  --namespace dorarion \
  --reuse-values \
  --set bootstrapAdmin.enabled=false \
  --values values.local.yaml \
  --wait \
  --timeout 15m
```

Ingress Controller나 인증서 관리자의 방식에 따라 TLS Secret을 구성하세요. TLS 개인 키를 values 파일에 넣지 마세요.

10. Dorarion 업그레이드

업그레이드 전에 다음을 수행합니다.

1. PostgreSQL을 백업하고 복원이 가능한지 시험합니다.
2. 정확한 `SECRET_ENCRYPTION_KEY`와 키 참조를 백업합니다.
3. 릴리스 노트와 마이그레이션 호환성 정책을 읽습니다.
4. 새로운 불변 차트 및 애플리케이션 버전을 선택합니다.

관리자 bootstrap을 비활성화한 상태로 업그레이드합니다.

```
helm upgrade dorarion \
  . \
  --version NEW_CHART_VERSION \
  --namespace dorarion \
  --reuse-values \
  --set bootstrapAdmin.enabled=false \
  --wait \
  --timeout 15m
```

pre-upgrade migration은 데이터베이스를 앞으로 변경합니다. Helm 릴리스를 롤백해도 데이터베이스 마이그레이션은 되돌아가지 않습니다.

11. 제거 범위

Helm 릴리스를 제거합니다.

```
helm uninstall dorarion --namespace dorarion
```

외부 PostgreSQL 데이터베이스와 수동으로 만든 `dorarion-secrets` Secret은 삭제되지 않습니다. 재설치나 복원 가능성이 있으면 둘 다 보존하세요. 남아 있는 모든 네임스페이스 리소스를 의도적으로 제거할 때만 네임스페이스를 삭제하세요.

PostgreSQL 예제를 배포했다면 이 리소스도 Helm 릴리스와 별개입니다. 일반적인 `Delete` reclaim policy를 사용하는 StorageClass에서는 영구 볼륨 클레임을 삭제할 때 데이터베이스도 삭제됩니다.

```
kubectl -n dorarion delete deployment,service dorarion-postgresql
kubectl -n dorarion delete persistentvolumeclaim dorarion-postgresql
kubectl -n dorarion delete secret dorarion-postgresql
```

예제 데이터베이스를 의도적으로 폐기하고 필요한 백업을 확보한 경우가 아니면 영구 볼륨 클레임 삭제 명령을 실행하지 마세요.

빠른 요약

- Kubernetes 컨텍스트와 버전을 확인했습니다.
- Docker Hub에 게시된 OCI 차트를 확인했습니다.
- 외부 PostgreSQL 데이터베이스와 백업을 준비했습니다.
- 또는 테스트용 단일 노드 PostgreSQL 예제를 배포했습니다.
- 네임스페이스와 필수 Kubernetes Secret을 만들었습니다.
- 첫 설치에서만 bootstrap을 활성화하여 Dorarion을 설치했습니다.
- 미리 만든 관리자 비밀번호를 비밀번호 관리자에 저장했습니다. bootstrap 로그에는 비밀번호가 없습니다.
- Deployment, readiness, 브라우저 접근을 확인했습니다.
- 이후 업그레이드에서는 bootstrap을 비활성화합니다.

공식 참고 자료

- Dorarion Helm 소스
- Docker Hub의 Dorarion 차트
- Helm 설치
- Helm OCI Registry
- kubectl 설치

설치와 업그레이드 — reviewed core

이 설정은 검토된 1.0 이전 core 소스를 기준으로 합니다. 해당 소스로 빌드한 이미지를 선택하세요. 기존 v0.0.3 레지스트리 태그만으로 보안 개선 포함 여부를 확인할 수 없습니다.

1.0 이전 버전입니다. 선택한 이미지의 API 계약을 기준으로 연동하세요. 검토된 소스는 CI/CD, 운영 작업, 장애 및 일간·주간 활동 보고를 지원합니다.

- `JWT_SECRET`(32바이트 이상)과 `SECRET_ENCRYPTION_KEY`(무작위 32바이트의 Base64)를 한 번 생성하고 Git 밖에 보관하세요.
- API 풀 기본값은 15개입니다. 복제본마다 워커 연결 6개와 임시 마이그레이션 연결도 확보하세요.
- Compose는 내부 `postgres` 호스트만 명시적으로 신뢰합니다. 외부 PostgreSQL은 `sslmode=verify-full`과 신뢰할 CA가 필요합니다.
- 외부 자격 증명은 `env://DORARION_EXT_<NAME>`을 사용합니다. core 비밀 환경 변수는 항상 차단됩니다.
- Argo CD는 애플리케이션별 회전 토큰과 활성 애플리케이션·연동이 필요합니다. 장애 자동화는 저장소 범위가 지정된 incident source를 사용합니다.

`/health/live`는 프로세스, `/health/ready`는 PostgreSQL을 확인합니다. 워커 상태는 인증이 필요한 `/health/workers`에서 따로 확인합니다. Prometheus에는 Secret에 보관한 ASCII 32–512자 `DORARION_METRICS_SCRAPE_TOKEN`을 사용하고 `/metrics`에만 전달하세요. 전달된 클라이언트 주소를 신뢰하기 전에 프록시 네트워크를 명시하세요.

전진 마이그레이션 전에 PostgreSQL과 정확한 암호화 키를 백업하세요. 마이그레이션 Job 또는 서비스를 한 번 실행하고 서버 시작 시 마이그레이션은 끕니다. 첫 로그인에서 생성된 관리자 비밀번호를 변경하고 업그레이드 후 Argo 애플리케이션 토큰을 회전하세요. 감사 보존 기간은 365일보다 짧을 수 없습니다. 주간 보고는 발송 전 완료된 현지 날짜 7일이며 수동 발송 재시도에는 같은 Idempotency-Key를 사용합니다. 장애 재개는 명시해야 합니다. 데모 경로는 제거되었고 알 수 없는 요청 필드는 거부되므로 이미지에 포함된 API 계약을 확인하세요. 운영 배포 전에 백업 복원, 공급자 전송과 readiness를 검증하세요.

Helm: `database.connectionPoolSize=15`, `database.statementTimeoutMs=30000`, `database.trustedHosts=[]`, `security.secretEnvAllowlist=[]`, `security.trustedProxies=[]`, `security.reportRecipientDomains=[]`, `security.readmeImageOrigins=[]`. Optional monitoring credentials belong in the existing Secret; select their key with `security.metricsScrapeTokenSecretKey`. Hook Jobs receive the same policy and `extraEnvFrom` as the server.